

PROJECT BASED EXPERIMENTAL LEARNING

COURSE – NO SQL DATABASE

PROJECT REPORT

TOPIC – MONGODB CRUD OPERATIONS

SUBMITTED TO	Mr. Harendra Singh
SUBMITTED BY	Harshit Kumar Srivastava
ROLL NO.	1250258215
CLASS	DS23
UNIVERSITY	Babu Banarasi Das University, Lucknow

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my mentor, **Mr. Harendra Singh**, for providing guidance and support during this project. This project helped me understand the practical use of MongoDB and the implementation of CRUD (Create, Read, Update and Delete) operations on a student dataset. I am also thankful to Babu Banarasi Das University, Lucknow, for providing the academic environment and resources required to complete this practical work.

ABSTRACT

This project presents the practical implementation of CRUD operations using MongoDB, a NoSQL document-oriented database. A database named **college** and a collection named **students** were created. Student records were inserted and then accessed using different read and query operations. Update operations were performed using operators such as **\$set** and **\$inc**, while delete operations were performed using **deleteOne()** and **deleteMany()**. The project demonstrates how MongoDB can be used to manage flexible document-based data through MongoDB Shell.

Keywords: MongoDB, NoSQL, CRUD, MongoDB Shell, Students Collection, Documents

INTRODUCTION

MongoDB is a NoSQL, document-oriented database that stores data in flexible, JSON-like documents using BSON. Unlike traditional relational databases, MongoDB does not require a fixed table schema for its documents. CRUD represents the four basic database operations: Create, Read, Update and Delete.

In this project, a **college** database and **students** collection were created. Student information such as roll number, name, department, semester, marks and city was stored as documents. The practical then demonstrated insertion, retrieval, filtering, sorting, updating and deletion of records.

The original practical contains 22 student documents and verifies the output after each major operation. After the update and delete operations, the final collection contains 17 documents.

METHODOLOGY

The project was completed through the following workflow:

- Install and use MongoDB Community Server, MongoDB Shell (mongosh) and MongoDB Compass.
- Create the **college** database and **students** collection.
- Insert multiple student documents using **insertMany()** and one additional document using **insertOne()**.
- Read records using **find()** and **findOne()**, with filtering conditions and comparison operators.
- Use multiple conditions, the **\$in** operator, projection, sorting and limiting.
- Update records using **updateOne()**, **\$set**, **\$inc** and **updateMany()**.
- Delete records using **deleteOne()** and **deleteMany()**.
- Verify the final collection using **find()** and **countDocuments()**.

IMPLEMENTATION DETAILS

The implementation follows the same operations and data used in the submitted MongoDB practical.

1. Create database and collection

```
use college
db.createCollection("students")
```

The database **college** and collection **students** were created successfully.

2. Create operation – insert multiple documents

```
db.students.insertMany([
  { roll_no: 1, name: "Harshit Srivastava", department: "CSE", semester: 5, marks: 82, city: "Lucknow" },
  { roll_no: 2, name: "Devansh Singh", department: "IT", semester: 3, marks: 74, city: "Kanpur" },
  { roll_no: 3, name: "Gauri Awasthi", department: "CSE", semester: 5, marks: 88, city: "Lucknow" },
  ...
])
```

The practical inserted 21 documents using **insertMany()**. One additional student was then inserted using **insertOne()**, bringing the collection to 22 documents.

3. Read operation – display and search

```
db.students.find().pretty()
db.students.findOne({ name: "Gauri Awasthi" })
db.students.find({ department: "CSE" })
```

The find operations displayed all documents, a single student record, and students belonging to the CSE department.

4. Query operators

```
db.students.find({ marks: { $gt: 80 } })
db.students.find({ department: "CSE", city: "Lucknow" })
db.students.find({ department: { $in: ["IT", "ECE"] } })
```

These queries demonstrated a comparison operator, multiple conditions and the **\$in** operator.

IMPLEMENTATION DETAILS (CONTINUED)

5. Projection, sorting and limiting

```
db.students.find({}, { name: 1, marks: 1, _id: 0 })
.sort({ marks: -1 })
.limit(5)
```

This operation displayed the top five scorers by selecting only name and marks, sorting marks in descending order and limiting the result to five documents.

6. Update operation

```
db.students.updateOne(
  { name: "Harshit Srivastava" },
  { $set: { marks: 86 } }
)

db.students.updateOne(
  { name: "Harshita Singh" },
  { $inc: { marks: 5 } }
)
```

Harshit Srivastava's marks were changed from 82 to 86 using **\$set**. Harshita Singh's marks increased from 67 to 72 using **\$inc**.

7. Update multiple documents

```
db.students.updateMany(
  { city: { $ne: "Lucknow" } },
  { $set: { hostel: true } }
)

db.students.updateMany(
  { department: "IT", semester: 3 },
  { $set: { semester: 4 } }
)
```

A hostel field was added to 15 students living outside Lucknow. Seven IT students from semester 3 were promoted to semester 4.

8. Delete operation

```
db.students.deleteOne({ name: "Kunal Yadav" })

db.students.deleteMany({ marks: { $lt: 60 } })
```

One specific student record was deleted using **deleteOne()**. Four students with marks below 60 were deleted using **deleteMany()**.

RESULTS

All CRUD operations were successfully performed on the **students** collection. The practical verified the outputs after insertion, searching, filtering, updating and deletion.

The top five scorers before the update/delete stage were:

Name	Marks
Ananya Srivastava	95
Riya Sharma	93
Ayushman Mishra	91
Anchal Gupta	89
Gauri Awasthi	88

After all update and delete operations, the final collection contained **17 documents**. The final verification used `find().pretty()` and `countDocuments()`.

Final CRUD verification:

- Create: Student documents were inserted successfully.
- Read: Documents were retrieved using `find()`, `findOne()` and filtered queries.
- Update: Individual and multiple documents were modified successfully.
- Delete: Individual and multiple documents were deleted successfully.

CONCLUSION

This project successfully demonstrates the practical use of MongoDB for managing student data. The CRUD operations were implemented step by step on the **students** collection. Different query techniques, update operators and deletion methods were also used. The project provides a clear practical understanding of document-based database operations in MongoDB.

The final verification confirmed that the operations were executed successfully and that the collection contained 17 documents after all modifications.

FUTURE SCOPE

- Develop a complete Student Management System using a web or desktop interface.
- Add authentication and role-based access for administrators and students.
- Add more student-related fields and collections such as courses, attendance and fees.
- Connect MongoDB with Python, Java or a web application for real-time data management.
- Add advanced reporting, search and analytics features.

REFERENCES

- MongoDB documentation and MongoDB Shell concepts used for CRUD operations.
- MongoDB Community Server / mongosh practical environment.
- MongoDB Compass for database and collection management.

THANK YOU!